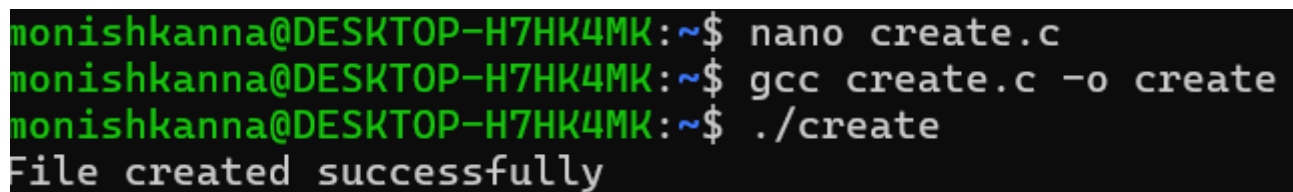


1. Write a C program to create a new file using the open() system call. What flags and permissions are required?

**CODE**

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main() {
    int fd = open("newfile.txt", O_CREAT | O_WRONLY, 0644);
    if (fd == -1) {
        perror("open failed");
        return 1;
    }
    printf("File created successfully\n");
    close(fd);
    return 0;
}
```

**OUTPUT**



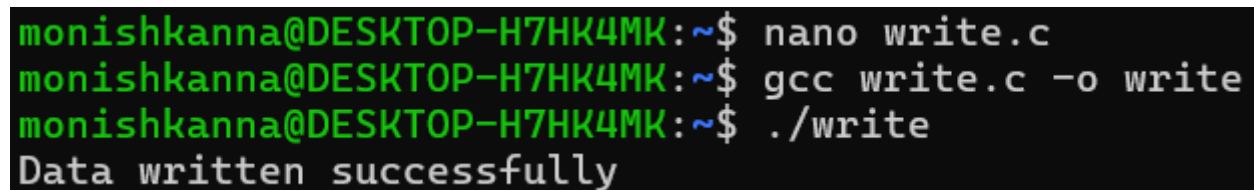
```
monishkanna@DESKTOP-H7HK4MK:~$ nano create.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc create.c -o create
monishkanna@DESKTOP-H7HK4MK:~$ ./create
File created successfully
```

2. Implement a program that writes "Hello, Linux!" to a file using write(). How do you handle errors?

**CODE**

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main() {
    int fd = open("hello.txt", O_CREAT | O_WRONLY, 0644);
    if (fd == -1) {
        perror("open failed");
        return 1;
    }
    if (write(fd, "Hello, Linux!\n", 14) == -1) {
        perror("write");
        return 1;
    }
    printf("Data written successfully\n");
    close(fd);
    return 0;
}
```

**OUTPUT**



```
monishkanna@DESKTOP-H7HK4MK:~$ nano write.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc write.c -o write
monishkanna@DESKTOP-H7HK4MK:~$ ./write
Data written successfully
```

3. Write a C program to read the contents of a file using read() and display it on the console.

**CODE**

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
```

```
int main() {
    char buffer[100];
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) {
        perror("open failed");
        return 1;
    }
    int bytes = read(fd, buffer, sizeof(buffer));
    write(STDOUT_FILENO, buffer, bytes);
    close(fd);
    return 0;
}
```

#### OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano read.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc read.c -o read
monishkanna@DESKTOP-H7HK4MK:~$ ./read
Hello, Linux!
```

4. Demonstrate how to use lseek() to move the file pointer to the beginning, middle, and end of a file.

#### CODE

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main() {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    printf("Beginning position: %ld\n", lseek(fd, 0, SEEK_SET));
    printf("End position: %ld\n", lseek(fd, 0, SEEK_END));
    close(fd);
    return 0;
}
```

#### OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano lseek.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc lseek.c -o lseek
monishkanna@DESKTOP-H7HK4MK:~$ ./lseek
Beginning: 0
End: 14
Middle: 7
```

5. Write a program to copy the contents of one file to another using only system calls (open, read, write, close).

#### CODE

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main() {
    char buffer[1024];
    int src = open("hello.txt", O_RDONLY);
    int dest = open("copy.txt", O_CREAT | O_WRONLY, 0644);
```

```

if (src == -1 || dest == -1) {
    perror("open");
    return 1;
}
int bytes;
while ((bytes = read(src, buffer, sizeof(buffer))) > 0) {
    write(dest, buffer, bytes);
}
printf("File copied successfully\n");
close(src);
close(dest);
return 0;
}

```

#### OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ nano copy.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc copy.c -o copy
monishkanna@DESKTOP-H7HK4MK:~$ ./copy
File copied successfully

```

6. Show how to change the permissions of a file using the chmod() system call.

#### CODE

```

#include <stdio.h>
#include <sys/stat.h>
#include <unistd.h>
int main() {
    int result;
    result = chmod("test.txt", 0777);
    if (result == 0) {
        printf("Permissions changed successfully.\n");
    } else {
        perror("chmod failed");
    }
    return 0;
}

```

#### OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ touch test.txt
monishkanna@DESKTOP-H7HK4MK:~$ ls -l test.txt
-rw-r--r-- 1 monishkanna monishkanna 0 Mar  1 17:12 test.txt
monishkanna@DESKTOP-H7HK4MK:~$ gcc chmod_sys.c -o chmod_sys
monishkanna@DESKTOP-H7HK4MK:~$ ./chmod_sys
Permissions changed successfully
monishkanna@DESKTOP-H7HK4MK:~$ ls -l test.txt
-rwxrwxrwx 1 monishkanna monishkanna 0 Mar  1 17:12 test.txt

```

7. Write a C program to delete a file using the unlink() system call.

#### CODE

```

#include <stdio.h>
#include <unistd.h>
int main() {
    if (unlink("delete.txt") == 0) {
        printf("File deleted successfully.\n");
    } else {perror("unlink failed");}
    return 0; }

```

**OUTPUT**

```
monishkanna@DESKTOP-H7HK4MK:~$ nano unlink_sys.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc unlink_sys.c -o unlink_sys
monishkanna@DESKTOP-H7HK4MK:~$ touch delete.txt
monishkanna@DESKTOP-H7HK4MK:~$ ./unlink_sys
File deleted successfully
```

8. Implement a program that creates a directory using `mkdir()` and removes it using `rmdir()`.

**CODE**

```
#include <stdio.h>
#include <sys/stat.h>
#include <unistd.h>
int main() {
    if (mkdir("mydir", 0777) == 0)
        printf("Directory created successfully.\n");
    else
        perror("mkdir failed");
    if (rmdir("mydir") == 0)
        printf("Directory removed successfully.\n");
    else
        perror("rmdir failed");
    return 0;
}
```

**OUTPUT**

```
monishkanna@DESKTOP-H7HK4MK:~$ nano dir_sys.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc dir_sys.c -o dir_sys
monishkanna@DESKTOP-H7HK4MK:~$ ./dir_sys
Directory created successfully.
Directory removed successfully.
```

9. Write a program to list all files in a directory using `opendir()`, `readdir()`, and `closedir()`.

**CODE**

```
#include <stdio.h>
#include <dirent.h>
int main() {
    DIR *dir;
    struct dirent *entry;
    dir = opendir(".");
    if (dir == NULL) {
        perror("opendir failed");
        return 1;
    }
    while ((entry = readdir(dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }
    closedir(dir);
    return 0;
}
```

## OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano listfiles.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc listfiles.c -o listfiles
monishkanna@DESKTOP-H7HK4MK:~$ ./listfiles
listfiles.c
dir_sys.c
p2
p2.c
a.out
copy.c
.sudo_as_admin_successful
.bash_history
lseek
.bashrc
..
.bash_logout
read.c
.motd_shown
chmod_sys.c
dir_sys
read
test
.profile
lseek.c
hello.txt
test.c
unlink_sys.c
.
test.txt
.local
chmod_sys
copy.txt
write.c
write
p1.c
create
create.c
.cache
copy
unlink_sys
newfile.txt
p1
listfiles
```

10. Demonstrate how to use stat() to display file metadata (size, permissions, modification time).

## CODE

```
#include <stdio.h>
#include <sys/stat.h>
#include <time.h>
int main() {
    struct stat s;
    if(stat("test.txt",&s)==0)
    {
        printf("File Size : %ld bytes\n",s.st_size);
        printf("Permissions : %o\n",s.st_mode & 0777);
        printf("Last Modified : %s",ctime(&s.st_mtime));
    }
    else
    { perror("stat failed");}
    return 0;
}
```

## OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ nano stat_sys.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc stat_sys.c -o stat_sys
monishkanna@DESKTOP-H7HK4MK:~$ ./stat_sys
File Size : 0 bytes
Permissions : 777
Last Modified : Sun Mar 1 17:12:35 2026

```

11. Write a C program that demonstrates a race condition when two processes write to the same file simultaneously. How can you prevent this using file locks?

#### CODE

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
int main()
{
    int fd;
    fd = open("race.txt", O_WRONLY | O_CREAT | O_APPEND, 0666);
    for(int i=0;i<5;i++){
        write(fd, "Process writing\n", 16);
        sleep(1);}
    close(fd);
    return 0;
}

```

#### OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ nano race.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc race.c -o race
monishkanna@DESKTOP-H7HK4MK:~$ ./race
monishkanna@DESKTOP-H7HK4MK:~$ cat race.txt
Process writing
Process writing
Process writing
Process writing
Process writing

```

12. Implement a program that atomically appends data to a file from multiple processes. Explain how you ensure atomicity.

#### CODE

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main(){
    int fd;
    fd=open("atomic.txt",O_WRONLY | O_CREAT | O_APPEND,0666);
    write(fd,"Atomic Write\n",13);
    close(fd);
    return 0;}

```

#### OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ nano atomic.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc atomic.c -o atomic
monishkanna@DESKTOP-H7HK4MK:~$ ./atomic
monishkanna@DESKTOP-H7HK4MK:~$ cat atomic.txt
Atomic Write

```

13. Use lseek() to create a sparse file. Show how to verify the file's actual disk usage versus its reported size.

#### CODE

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
int main()
{
    int fd;
    fd=open("sparse.txt",O_WRONLY | O_CREAT,0666);
    lseek(fd,1024*1024,SEEK_SET);
    write(fd,"END",3);
    close(fd);
    printf("Sparse file created\n");
    return 0;
}
```

#### OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano sparse.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sparse.c -o sparse
monishkanna@DESKTOP-H7HK4MK:~$ ./sparse
Sparse file created
monishkanna@DESKTOP-H7HK4MK:~$ ls -lh sparse.txt
-rw-r--r-- 1 monishkanna monishkanna 1.1M Mar  4 05:50 sparse.txt
monishkanna@DESKTOP-H7HK4MK:~$ du -h sparse.txt
4.0K    sparse.txt
```

14. Write a program that uses fcntl() to set a write lock on a region of a file. How do you handle lock contention between processes?

#### CODE

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd = open("lockfile.txt", O_RDWR | O_CREAT, 0666);
    struct flock lock;
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 20;
    lock.l_pid = getpid();
    printf("Trying to acquire write lock...\n");
    if (fcntl(fd, F_SETLK, &lock) == -1) {
        perror("Lock failed (another process holds lock)"); }
    else { printf("Write lock acquired.\n");
        sleep(10); }
    close(fd);
    return 0;}
```

#### OUTPUT

```
monishkanna@DESKTOP-H7HK4MK:~$ nano write_lock.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc write_lock.c -o write_lock
monishkanna@DESKTOP-H7HK4MK:~$ ./write_lock
Trying to acquire write lock...
Write lock acquired.
monishkanna@DESKTOP-H7HK4MK:~$ ./write_lock
Trying to acquire write lock...
Lock failed (another process holds lock): Resource temporarily unavailable
```

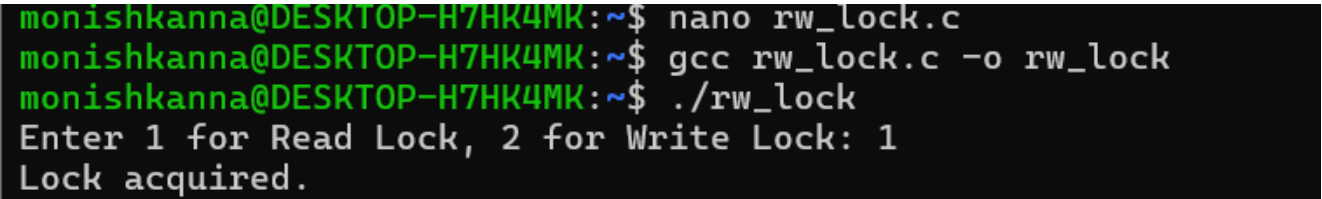
15. Demonstrate how to use `fcntl()` to implement a read lock and a write lock, and explain the difference in behavior when multiple processes access the file.

**CODE**

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd = open("lockfile.txt", O_RDWR);
    struct flock lock;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0; // Whole file
    lock.l_pid = getpid();
    printf("Enter 1 for Read Lock, 2 for Write Lock: ");
    int choice;
    scanf("%d", &choice);
    if(choice == 1)
        lock.l_type = F_RDLCK;
    else
        lock.l_type = F_WRLCK;

    if(fcntl(fd, F_SETLK, &lock) == -1)
        perror("Lock failed");
    else {
        printf("Lock acquired.\n");
        sleep(10);
    }
    close(fd);
    return 0;
}
```

**OUTPUT**



```
monishkanna@DESKTOP-H7HK4MK:~$ nano rw_lock.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc rw_lock.c -o rw_lock
monishkanna@DESKTOP-H7HK4MK:~$ ./rw_lock
Enter 1 for Read Lock, 2 for Write Lock: 1
Lock acquired.
```

16. Create a program that uses `lseek()` to read and write at random offsets in a file. How do you ensure data consistency?

**CODE**

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
int main() {
    int fd = open("random.txt", O_RDWR | O_CREAT | O_TRUNC, 0666);
    if(fd < 0) {
        perror("open failed");
        return 1;
    }
    lseek(fd, 10, SEEK_SET);
    write(fd, "HELLO", 5);
}
```



```

lseek(fd, 10, SEEK_SET);
char buffer[6];
read(fd, buffer, 5);
buffer[5] = '\0';
printf("File Content: %s\n", buffer);
close(fd);
return 0;
}

```

#### OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ nano random_access.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc random_access.c -o random_access
monishkanna@DESKTOP-H7HK4MK:~$ ./random_access
File Content: HELLO

```

17. Write a C program that uses advisory file locking (fcntl) to synchronize access between a parent and child process. What happens if one process ignores the lock?

#### CODE

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    int fd = open("sync.txt", O_RDWR | O_CREAT, 0666);
    if(fork() == 0) {
        struct flock lock = {F_WRLCK, SEEK_SET, 0, 0, getpid()};
        fcntl(fd, F_SETLKW, &lock);
        printf("Child acquired lock\n");
        sleep(5);
    } else {
        sleep(1);
        struct flock lock = {F_WRLCK, SEEK_SET, 0, 0, getpid()};
        fcntl(fd, F_SETLKW, &lock);
        printf("Parent acquired lock\n");
        wait(NULL); }
    close(fd);
    return 0;
}

```

#### OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ nano sync_lock.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sync_lock.c -o sync_lock
monishkanna@DESKTOP-H7HK4MK:~$ ./sync_lock
Child acquired lock
Parent acquired lock

```

18. Implement a program that tests for deadlock scenarios using file locks. How can you detect and resolve deadlocks?

#### CODE

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd = open("deadlock.txt", O_RDWR | O_CREAT, 0666);

```

```

if(fd < 0) {
    perror("open failed");
    exit(1);
} if(fork() == 0) {
    struct flock lock1, lock2;
    lock1.l_type = F_WRLCK;
    lock1.l_whence = SEEK_SET;
    lock1.l_start = 0;
    lock1.l_len = 10;
    lock1.l_pid = getpid();
    printf("Child locking region 1...\n");
    fcntl(fd, F_SETLKW, &lock1);
    sleep(2);
    lock2.l_type = F_WRLCK;
    lock2.l_whence = SEEK_SET;
    lock2.l_start = 10;
    lock2.l_len = 10;
    lock2.l_pid = getpid();
    printf("Child trying to lock region 2...\n");
    if(fcntl(fd, F_SETLKW, &lock2) == -1)
        perror("Child deadlock detected");
    exit(0);
} else {
    struct flock lock1, lock2;
    lock1.l_type = F_WRLCK;
    lock1.l_whence = SEEK_SET;
    lock1.l_start = 10;
    lock1.l_len = 10;
    lock1.l_pid = getpid();
    printf("Parent locking region 2...\n");
    fcntl(fd, F_SETLKW, &lock1);
    sleep(2);
    lock2.l_type = F_WRLCK;
    lock2.l_whence = SEEK_SET;
    lock2.l_start = 0;
    lock2.l_len = 10;
    lock2.l_pid = getpid();
    printf("Parent trying to lock region 1...\n");
    if(fcntl(fd, F_SETLKW, &lock2) == -1)
        perror("Parent deadlock detected");
    wait(NULL);
}
close(fd);
return 0;
}

```

## OUTPUT

```

monishkanna@DESKTOP-H7HK4MK:~$ nano deadlock.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc deadlock.c -o deadlock
monishkanna@DESKTOP-H7HK4MK:~$ ./deadlock
Parent locking region 2...
Child locking region 1...
Child trying to lock region 2...
Parent trying to lock region 1...

^C
monishkanna@DESKTOP-H7HK4MK:~$ |

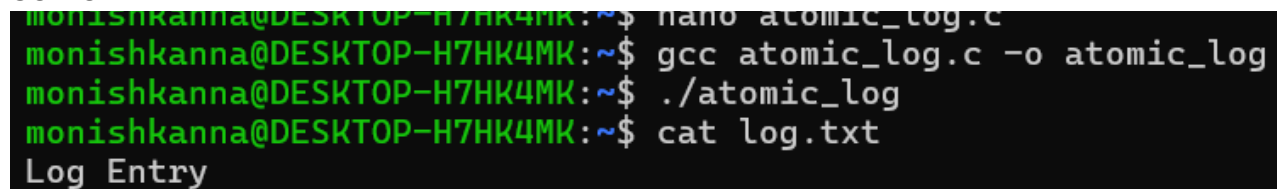
```

19. Show how to use O\_APPEND and file locks together to guarantee atomic writes to a log file from multiple processes.

**CODE**

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd = open("log.txt", O_WRONLY | O_CREAT | O_APPEND, 0666);
    struct flock lock = {F_WRLCK, SEEK_SET, 0, 0, getpid()};
    fcntl(fd, F_SETLKW, &lock);
    write(fd, "Log Entry\n", 10);
    lock.l_type = F_UNLCK;
    fcntl(fd, F_SETLK, &lock);
    close(fd);
    return 0;
}
```

**OUTPUT**



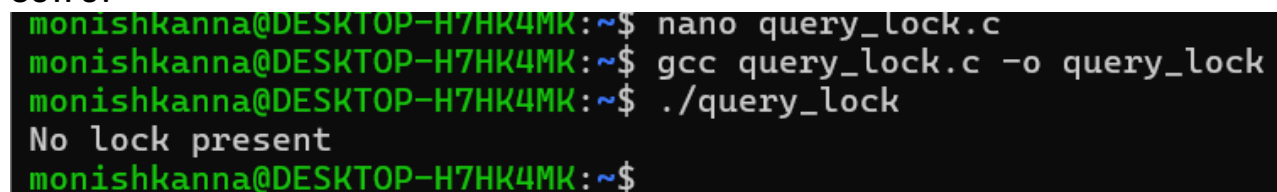
```
monishkanna@DESKTOP-H7HK4MK:~$ nano atomic_log.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc atomic_log.c -o atomic_log
monishkanna@DESKTOP-H7HK4MK:~$ ./atomic_log
monishkanna@DESKTOP-H7HK4MK:~$ cat log.txt
Log Entry
```

20. Write a program that uses fcntl() to query and display all current locks on a file. How do you interpret the results?

**CODE**

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd = open("lockfile.txt", O_RDWR);
    struct flock lock;
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;
    if(fcntl(fd, F_GETLK, &lock) == -1)
        perror("F_GETLK failed");
    else {
        if(lock.l_type == F_UNLCK)
            printf("No lock present\n");
        else
            printf("Locked by PID: %d\n", lock.l_pid);
    }
    close(fd);
    return 0;
}
```

**OUTPUT**



```
monishkanna@DESKTOP-H7HK4MK:~$ nano query_lock.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc query_lock.c -o query_lock
monishkanna@DESKTOP-H7HK4MK:~$ ./query_lock
No lock present
monishkanna@DESKTOP-H7HK4MK:~$
```